

Módulo: Subconsultas em T-SQL (SQL Server)

Curso: Banco de Dados Relacionais aplicado à Administração Tributária **Público-alvo:** Profissionais da Secretaria de Estado da Fazenda da Paraíba (SEFAZ-PB) **SGBD utilizado:** Microsoft SQL Server (T-SQL) — ferramenta: SQL Server Management Studio (SSMS) **Carga horária sugerida:** 4 horas (2h teoria + 2h prática) **Pré-requisitos:** domínio de **SELECT**, **JOIN**, **GROUP BY/HAVING** e das cláusulas DML/DQL básicas (ver [00 Comandos SQL.md](#))

Sumário

1. [Objetivos do Módulo](#)
 2. [O que é uma Subconsulta](#)
 3. [Classificação das Subconsultas](#)
 4. [Subconsultas Escalares](#)
 5. [Subconsultas na Cláusula WHERE](#)
 6. [EXISTS e NOT EXISTS](#)
 7. [Subconsultas Correlacionadas](#)
 8. [Subconsultas na Cláusula FROM \(Derived Tables\)](#)
 9. [Subconsultas na Cláusula HAVING](#)
 10. [APPLY — Subconsultas Correlacionadas como Junção](#)
 11. [O Perigo do NOT IN com NULL](#)
 12. [Subconsulta x JOIN x CTE: Quando Usar Cada Um](#)
 13. [Desempenho e Plano de Execução](#)
 14. [Cenário Integrado: Análise Fiscal Completa](#)
 15. [Boas Práticas](#)
 16. [Glossário](#)
 17. [Exercícios](#)
 18. [Referências](#)
-

1. Objetivos do Módulo

Ao final deste módulo, o participante será capaz de:

- Definir o que é uma subconsulta (*subquery*) e reconhecer em quais cláusulas do T-SQL ela pode aparecer (**SELECT**, **FROM**, **WHERE**, **HAVING**).
- Classificar subconsultas quanto ao **retorno** (escalar, de linha, de tabela) e quanto à **dependência** (independentes x correlacionadas).
- Escrever subconsultas com **IN**, **NOT IN**, **ANY/SOME**, **ALL**, **EXISTS** e **NOT EXISTS**, entendendo a semântica de cada operador.
- Identificar e evitar a armadilha clássica do **NOT IN** combinado com valores **NULL**.
- Reescrever subconsultas correlacionadas usando **CROSS APPLY/OUTER APPLY** quando apropriado.
- Comparar subconsultas com **JOIN** e com CTEs (**WITH**), escolhendo a abordagem mais legível e performática para cada cenário fiscal.

- Ler um plano de execução simples do SSMS para diagnosticar o custo de uma subconsulta correlacionada.

2. O que é uma Subconsulta

Uma **subconsulta** (*subquery*) é uma instrução **SELECT** completa, escrita entre parênteses, aninhada dentro de outra instrução T-SQL (**SELECT**, **INSERT**, **UPDATE**, **DELETE** ou até dentro de outra subconsulta). A consulta que contém a subconsulta é chamada de **consulta externa** (*outer query*); a consulta aninhada é a **consulta interna** (*inner query*).

```
SELECT razao_social
FROM dbo.contribuinte
WHERE id_contribuinte IN (
    SELECT id_emitente                -- consulta interna (subconsulta)
    FROM dbo.nfe
    WHERE valor_total > 50000.00
);                                   -- consulta externa
```

Por que usar subconsultas:

- Permitem expressar "**consultas dentro de consultas**" — por exemplo, "quais contribuintes emitiram notas acima da média geral" exige primeiro calcular a média (subconsulta) para depois filtrar (consulta externa).
- Tornam certas perguntas de negócio mais legíveis do que a alternativa equivalente com **JOIN** + **GROUP BY**.
- São a base conceitual para recursos mais avançados como CTEs, **APPLY** e funções de janela.

Regras gerais do T-SQL:

- A subconsulta é sempre delimitada por parênteses (...).
- Uma subconsulta que retorna uma única coluna e um único valor pode ser usada onde uma expressão escalar é esperada (ex.: após =).
- Uma subconsulta não pode conter **ORDER BY**, exceto quando usada junto com **TOP**.
- Subconsultas podem ser aninhadas em múltiplos níveis, mas mais de 2–3 níveis geralmente prejudicam a legibilidade — nesses casos, uma CTE costuma ser preferível (ver [seção 12](#)).

3. Classificação das Subconsultas

Subconsultas podem ser classificadas sob duas óticas complementares:

3.1 Quanto ao formato do retorno

Tipo	Retorna	Onde costuma ser usada
Escalar	Uma única linha, uma única coluna (um valor)	Após =, >, <, em SELECT , em SET de UPDATE

Tipo	Retorna	Onde costuma ser usada
De linha (<i>row</i>)	Uma única linha, múltiplas colunas	Menos comum no T-SQL puro; usada em comparações de tupla em alguns SGBDs — no SQL Server, geralmente reescrita como múltiplas condições ou EXISTS
De tabela (<i>table</i>)	Múltiplas linhas, múltiplas colunas	Em FROM (derived table), com IN/ANY/ALL/EXISTS

3.2 Quanto à dependência da consulta externa

Tipo	Característica	Desempenho típico
Independente (<i>não correlacionada</i>)	Executada uma única vez , de forma isolada; não referencia colunas da consulta externa	Geralmente mais previsível — o otimizador pode calculá-la uma vez e reutilizar o resultado
Correlacionada	Referencia uma ou mais colunas da consulta externa; conceitualmente reexecutada para cada linha candidata da consulta externa	Pode ser cara em tabelas grandes se não houver índice de apoio — ver seção 7 e seção 13

Nota: dizer que uma subconsulta correlacionada "executa uma vez por linha" é a forma didática de explicar a semântica lógica do comando. Na prática, o **otimizador de consultas** do SQL Server frequentemente reescreve a subconsulta correlacionada como um **JOIN/APPLY** internamente, evitando a execução literal linha a linha. Ainda assim, o comportamento lógico (o resultado) é sempre equivalente a uma reavaliação por linha.

4. Subconsultas Escalares

Uma subconsulta escalar retorna **exatamente um valor** (uma linha, uma coluna) e pode ser usada em qualquer lugar onde uma expressão simples seria aceita.

4.1 No **SELECT** (coluna calculada)

```
SELECT
    n.chave_acesso,
    n.valor_total,
    (SELECT AVG(valor_total) FROM dbo.nfe) AS media_geral_notas
FROM dbo.nfe AS n;
```

Cada linha do resultado exibe o mesmo valor de **media_geral_notas** — a subconsulta é independente e é recalculada (ou reaproveitada pelo otimizador) para todas as linhas.

4.2 Comparando com um valor único no **WHERE**

```
SELECT razao_social, regime_tributario
FROM dbo.contribuinte
```

```
WHERE id_contribuinte = (  
    SELECT TOP (1) id_emitente  
    FROM dbo.nfe  
    ORDER BY valor_total DESC, id_nfe DESC  
);
```

Cuidado T-SQL: se a subconsulta escalar retornar **mais de uma linha** em tempo de execução, o SQL Server lança o erro:

```
Subquery returned more than 1 value. This is not permitted when the subquery  
follows =, !=, <, <=, >, >= or when the subquery is used as an expression.
```

Por isso, subconsultas escalares em comparações simples (=) devem sempre garantir unicidade — com **TOP (1) + ORDER BY**, agregações (**MAX**, **MIN**, **AVG**) ou filtros que naturalmente resultem em uma única linha (ex.: filtro por chave primária).

4.3 Em UPDATE

```
UPDATE c  
SET faturamento_declarado = (  
    SELECT SUM(n.valor_total)  
    FROM dbo.nfe AS n  
    WHERE n.id_emitente = c.id_contribuinte  
)  
FROM dbo.contribuinte AS c  
WHERE EXISTS (  
    SELECT 1  
    FROM dbo.nfe AS n  
    WHERE n.id_emitente = c.id_contribuinte  
);
```

Aqui a subconsulta do **SET** é **correlacionada** (referencia `dbo.contribuinte.id_contribuinte` da linha sendo atualizada), assim como a subconsulta do **WHERE**.

5. Subconsultas na Cláusula WHERE

Esta é a aplicação mais comum de subconsultas: filtrar a consulta externa com base em um conjunto de valores produzido pela consulta interna.

5.1 IN — pertence a um conjunto

```
SELECT razao_social  
FROM dbo.contribuinte  
WHERE id_contribuinte IN (  
    SELECT id_emitente
```

```
FROM dbo.nfe
WHERE situacao = 'CANCELADA'
);
```

Retorna contribuintes que emitiram **pelo menos uma** nota cancelada.

5.2 NOT IN — não pertence a um conjunto

```
SELECT razao_social
FROM dbo.contribuinte
WHERE id_contribuinte NOT IN (
    SELECT id_emitente
    FROM dbo.nfe
    WHERE situacao = 'CANCELADA'
);
```

Retorna contribuintes que **nunca** emitiram nota cancelada. **Atenção:** ver a armadilha do **NULL** na [seção 11](#) antes de usar **NOT IN** em produção.

5.3 ANY / SOME — compara com pelo menos um valor do conjunto

ANY e **SOME** são sinônimos no T-SQL; a condição é satisfeita se ela for verdadeira para **pelo menos um** valor retornado pela subconsulta.

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE c.id_contribuinte = ANY (
    SELECT id_emitente
    FROM dbo.nfe
    WHERE valor_total > 100000.00
);
```

Exemplo mais natural — contribuintes com pelo menos uma nota maior que **qualquer** nota emitida no regime **SIMPLES**:

```
SELECT razao_social
FROM dbo.contribuinte AS c
JOIN dbo.nfe AS n
    ON n.id_emitente = c.id_contribuinte
WHERE n.valor_total > ANY (
    SELECT valor_total
    FROM dbo.nfe AS n2
    JOIN dbo.contribuinte AS c2 ON c2.id_contribuinte = n2.id_emitente
    WHERE c2.regime_tributario = 'SIMPLES'
);
```

> **ANY (...)** equivale a "maior que o **menor** valor do conjunto" — ou seja, é suficiente superar apenas um dos valores.

5.4 **ALL** — compara com todos os valores do conjunto

A condição só é satisfeita se ela for verdadeira para **todos** os valores retornados pela subconsulta.

```
SELECT razao_social
FROM dbo.contribuinte AS c
JOIN dbo.nfe AS n
  ON n.id_emitente = c.id_contribuinte
WHERE n.valor_total > ALL (
  SELECT valor_total
  FROM dbo.nfe AS n2
  JOIN dbo.contribuinte AS c2 ON c2.id_contribuinte = n2.id_emitente
  WHERE c2.regime_tributario = 'SIMPLES'
);
```

> **ALL (...)** equivale a "maior que o **maior** valor do conjunto". Contribuintes retornados aqui emitiram uma nota mais valiosa do que **qualquer** nota do regime **SIMPLES**.

Operador	Equivalência lógica	Cuidado
= ANY (...)	Equivalente a IN (...)	—
<> ALL (...)	Equivalente a NOT IN (...) (mas sem a armadilha do NULL , se reescrito com EXISTS)	Prefira NOT EXISTS quando o conjunto pode ter NULL
> ANY (...)	Maior que o mínimo do conjunto	Se o conjunto vier vazio, a condição é FALSE
> ALL (...)	Maior que o máximo do conjunto	Se o conjunto vier vazio, a condição é TRUE (comportamento contraintuitivo — testar sempre)

6. **EXISTS** e **NOT EXISTS**

EXISTS testa apenas **se a subconsulta retorna alguma linha**, sem se importar com os valores retornados — por isso, é comum escrever **SELECT 1** ou **SELECT *** dentro da subconsulta, sem impacto de desempenho (o otimizador não materializa as colunas).

6.1 **EXISTS**

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE EXISTS (
```

```
SELECT 1
FROM dbo.nfe AS n
WHERE n.id_emitente = c.id_contribuinte
      AND n.situacao = 'CANCELADA'
);
```

Retorna contribuintes que possuem pelo menos uma nota cancelada — mesmo resultado do exemplo com **IN** da [seção 5.1](#), mas com uma diferença estrutural importante: aqui a subconsulta é **correlacionada** (referencia **c.id_contribuinte** da linha externa).

6.2 NOT EXISTS

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
          AND n.situacao = 'CANCELADA'
);
```

Retorna contribuintes que **nunca** emitiram nota cancelada — semanticamente equivalente ao **NOT IN** da [seção 5.2](#), mas **imune ao problema de NULL** (ver [seção 11](#)). Por esse motivo, **NOT EXISTS** é a forma **recomendada** de expressar "não existe" em T-SQL de produção.

6.3 EXISTS x IN: quando preferir cada um

Critério	IN	EXISTS
Coluna única, sem risco de NULL	Ambos funcionam igual	Ambos funcionam igual
Coluna com possibilidade de NULL (especialmente com NOT IN)	Risco de resultado vazio inesperado	Seguro
Múltiplas condições de correlação	Não se aplica diretamente	Natural — pode combinar várias colunas no WHERE interno
Legibilidade para "existe pelo menos um relacionado"	Boa	Geralmente mais clara para quem já pensa em termos relacionais
Desempenho	O otimizador do SQL Server, na prática, costuma gerar planos equivalentes para IN/EXISTS bem escritos	Idem

7. Subconsultas Correlacionadas

Uma subconsulta é **correlacionada** quando referencia uma ou mais colunas da consulta externa em sua cláusula **WHERE** (ou **JOIN**), tornando seu resultado dependente da linha atualmente avaliada pela consulta externa.

7.1 Exemplo — nota mais recente de cada contribuinte

```
SELECT
    c.razao_social,
    (
        SELECT MAX(n.data_emissao)
        FROM dbo.nfe AS n
        WHERE n.id_emitente = c.id_contribuinte          -- correlação
    ) AS ultima_emissao
FROM dbo.contribuinte AS c;
```

A subconsulta não pode ser executada de forma isolada — ela **depende** de **c.id_contribuinte**, que muda a cada linha da consulta externa.

7.2 Exemplo — contribuintes cuja última nota foi cancelada

```
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE 'CANCELADA' = (
    SELECT TOP (1) n.situacao
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
    ORDER BY n.data_emissao DESC
);
```

7.3 Correlação em **UPDATE/DELETE**

```
-- Marca como "INAPTO" contribuintes sem nenhuma nota fiscal emitida nos últimos
24 meses
UPDATE dbo.contribuinte
SET situacao_cadastral = 'INAPTO'
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.nfe AS n
    WHERE n.id_emitente = dbo.contribuinte.id_contribuinte
        AND n.data_emissao >= DATEADD(MONTH, -24, SYSUTCDATETIME())
);
```

Subconsultas correlacionadas em **UPDATE/DELETE** são o padrão mais seguro para expressar "altere/apague com base em uma condição de outra tabela", evitando os efeitos colaterais de um **JOIN** malformado em

UPDATE ... FROM.

8. Subconsultas na Cláusula FROM (Derived Tables)

Uma subconsulta usada na cláusula FROM é chamada de **tabela derivada** (*derived table*). Ela é tratada como se fosse uma tabela temporária existente apenas durante a execução da consulta — e **exige um alias obrigatório** em T-SQL.

```
SELECT
    resumo.regime_tributario,
    resumo.total_contribuintes,
    resumo.valor_medio
FROM (
    SELECT
        c.regime_tributario,
        COUNT(DISTINCT c.id_contribuinte) AS total_contribuintes,
        AVG(n.valor_total) AS valor_medio
    FROM dbo.contribuinte AS c
    JOIN dbo.nfe AS n
        ON n.id_emitente = c.id_contribuinte
    GROUP BY c.regime_tributario
) AS resumo -- alias obrigatório
WHERE resumo.total_contribuintes > 10
ORDER BY resumo.valor_medio DESC;
```

Por que usar: permite aplicar um filtro (WHERE) ou uma nova agregação **sobre o resultado já agregado** de uma consulta anterior — algo que não é possível fazer diretamente em HAVING quando a condição envolve uma coluna derivada de forma mais complexa, ou quando o resultado intermediário precisa ser reutilizado várias vezes na mesma consulta (via JOIN com ele mesmo, por exemplo).

Alternativa moderna: para consultas com múltiplos níveis de tabela derivada, uma CTE (WITH nome AS (...)) costuma ser mais legível — ver comparação na [seção 12](#).

9. Subconsultas na Cláusula HAVING

Assim como o WHERE filtra linhas antes do agrupamento, o HAVING filtra **grupos** — e também aceita subconsultas, tipicamente para comparar um agregado do grupo com um valor calculado de forma independente.

```
SELECT
    c.regime_tributario,
    AVG(n.valor_total) AS media_regime
FROM dbo.contribuinte AS c
JOIN dbo.nfe AS n
    ON n.id_emitente = c.id_contribuinte
GROUP BY c.regime_tributario
HAVING AVG(n.valor_total) > (
```

```
SELECT AVG(valor_total) FROM dbo.nfe
);
```

Retorna apenas os regimes tributários cuja média de valor por nota supera a média **geral** de todas as notas — uma pergunta que não poderia ser respondida só com **WHERE**, porque a média geral é um agregado calculado sobre toda a tabela, não sobre cada grupo.

10. APPLY — Subconsultas Correlacionadas como Junção

CROSS APPLY e **OUTER APPLY** são extensões do T-SQL (sem equivalente direto no padrão ANSI) que permitem "juntar" cada linha da tabela externa com o resultado de uma subconsulta correlacionada **que retorna múltiplas colunas ou múltiplas linhas** — algo que uma subconsulta escalar tradicional não suporta.

10.1 CROSS APPLY — equivalente a um INNER JOIN correlacionado

```
SELECT
    c.razao_social,
    top3.chave_acesso,
    top3.valor_total
FROM dbo.contribuinte AS c
CROSS APPLY (
    SELECT TOP (3) n.chave_acesso, n.valor_total
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
    ORDER BY n.valor_total DESC
) AS top3;
```

Para **cada contribuinte**, retorna as **3 notas de maior valor** — uma consulta impossível de escrever com uma subconsulta escalar tradicional (que só devolve um valor), e trabalhosa de expressar só com **JOIN** + função de janela sem repetir lógica.

10.2 OUTER APPLY — equivalente a um LEFT JOIN correlacionado

```
SELECT
    c.razao_social,
    ultima.chave_acesso,
    ultima.data_emissao
FROM dbo.contribuinte AS c
OUTER APPLY (
    SELECT TOP (1) n.chave_acesso, n.data_emissao
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
    ORDER BY n.data_emissao DESC
) AS ultima;
```

Diferente de **CROSS APPLY**, o **OUTER APPLY** **preserva** contribuintes sem nenhuma nota fiscal (as colunas **ultima.*** vêm como **NULL**), da mesma forma que um **LEFT JOIN** preserva o lado esquerdo.

Cláusula	Comportamento quando a subconsulta não retorna linhas
CROSS APPLY	Descarta a linha externa (como INNER JOIN)
OUTER APPLY	Mantém a linha externa com NULL nas colunas da subconsulta (como LEFT JOIN)

11. O Perigo do **NOT IN** com **NULL**

Este é um dos erros mais comuns — e mais silenciosos — envolvendo subconsultas em T-SQL.

```
-- id_destinatario aceita NULL (ex.: consumidor nao identificado)
SELECT razao_social
FROM dbo.contribuinte
WHERE id_contribuinte NOT IN (
    SELECT id_destinatario
    FROM dbo.nfe
);
```

Se a subconsulta retornar sequer um **NULL** entre os valores de **id_destinatario**, a consulta externa inteira **não retorna nenhuma linha** — mesmo que existam contribuintes claramente ausentes da lista. Isso ocorre porque, em lógica de três valores do SQL (**TRUE/FALSE/UNKNOWN**), comparar qualquer valor com **NULL** usando **<>** resulta em **UNKNOWN**, e o **NOT IN** internamente é avaliado como uma cadeia de **AND (valor <> x1) AND (valor <> x2) AND ...** — bastando um único **UNKNOWN** para que toda a condição deixe de ser **TRUE**.

Formas seguras de evitar o problema:

```
-- Opção 1: filtrar o NULL explicitamente dentro da subconsulta
SELECT razao_social
FROM dbo.contribuinte
WHERE id_contribuinte NOT IN (
    SELECT id_destinatario
    FROM dbo.nfe
    WHERE id_destinatario IS NOT NULL
);

-- Opção 2 (recomendada): usar NOT EXISTS, imune ao problema
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.nfe AS n
    WHERE n.id_destinatario = c.id_contribuinte
);
```

Regra prática para a equipe: em rotinas fiscais de produção, prefira sempre **NOT EXISTS** a **NOT IN** quando houver qualquer possibilidade de **NULL** na coluna da subconsulta — o que, na dúvida, deve ser tratado como "sempre possível", a menos que a coluna tenha **NOT NULL** garantido por constraint.

12. Subconsulta x JOIN x CTE: Quando Usar Cada Um

As três abordagens frequentemente produzem o **mesmo resultado lógico**, mas comunicam intenções diferentes e nem sempre têm o mesmo desempenho.

```
-- (A) Subconsulta com EXISTS
SELECT c.razao_social
FROM dbo.contribuinte AS c
WHERE EXISTS (
    SELECT 1 FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
);

-- (B) JOIN equivalente (exige DISTINCT para não duplicar contribuintes com várias notas)
SELECT DISTINCT c.razao_social
FROM dbo.contribuinte AS c
JOIN dbo.nfe AS n
    ON n.id_emitente = c.id_contribuinte;

-- (C) CTE equivalente
WITH contribuintes_com_nota AS (
    SELECT DISTINCT id_emitente AS id_contribuinte
    FROM dbo.nfe
)
SELECT c.razao_social
FROM dbo.contribuinte AS c
JOIN contribuintes_com_nota AS ccn
    ON ccn.id_contribuinte = c.id_contribuinte;
```

Critério	Subconsulta (EXISTS/IN)	JOIN	CTE (WITH)
Quando a intenção é filtrar por existência	Mais natural e direto	Exige DISTINCT /cuidado com duplicação	Funciona, mas é mais verboso para esse caso simples
Quando é preciso trazer colunas da tabela relacionada	Não serve (subconsulta não expõe colunas para o SELECT externo)	Ideal	Ideal
Reaproveitar o mesmo resultado	Repetiria a subconsulta em cada lugar	Repetiria o JOIN	Ideal — a CTE é definida uma vez e

Critério	Subconsulta (EXISTS/IN)	JOIN	CTE (WITH)
intermediário várias vezes na mesma consulta			referenciada quantas vezes for preciso
Consultas recursivas (ex.: hierarquia de contas contábeis)	Não suportado	Não suportado diretamente	Único que suporta (WITH ... AS (... UNION ALL ...))
Legibilidade em consultas com 3+ níveis de aninhamento	Degrada rapidamente	Não se aplica	Mantém legibilidade (blocos nomeados e sequenciais)

Regra prática: comece pela forma mais direta para a pergunta de negócio (**EXISTS** para "existe?", **JOIN** para "traga os dados relacionados"); migre para CTE quando a consulta precisar reutilizar um resultado intermediário mais de uma vez ou quando o aninhamento de subconsultas passar de dois níveis.

13. Desempenho e Plano de Execução

Subconsultas **não são, por definição, mais lentas** que **JOINS** — o otimizador de consultas do SQL Server frequentemente as reescreve internamente para planos equivalentes. Ainda assim, alguns pontos merecem atenção prática:

- **Índices na coluna de correlação:** uma subconsulta correlacionada (ou um **EXISTS**) que filtra por `n.id_emitente = c.id_contribuinte` se beneficia diretamente de um índice em `dbo.nfe(id_emitente)`. Sem esse índice, cada avaliação da subconsulta pode exigir uma varredura completa da tabela (*table scan*).
- **SELECT 1 em EXISTS:** não há ganho real em trocar **SELECT 1** por **SELECT *** ou por uma coluna específica dentro de **EXISTS** — o otimizador ignora a lista de colunas nesse contexto, pois só o fato de "existir linha" importa.
- **Ler o plano de execução (SSMS):** **Ctrl+M** (Include Actual Execution Plan) antes de rodar a consulta permite visualizar se uma subconsulta foi transformada em **Nested Loops**, **Hash Match** ou **Merge Join** — e se algum operador está fazendo *scan* em vez de *seek*.
- **TOP (1) sem ORDER BY determinístico:** ao usar **TOP (1)** dentro de uma subconsulta escalar (ex.: [seção 4.2](#)) sem uma coluna que garanta desempate único (como a chave primária como critério secundário), o resultado pode variar entre execuções caso haja empate no critério de ordenação principal.
- **Subconsultas correlacionadas em grande volume:** para tabelas fiscais com milhões de notas, compare sempre o desempenho da subconsulta correlacionada com a alternativa via **CROSS APPLY/OUTER APPLY** ([seção 10](#)) ou função de janela (`ROW_NUMBER() OVER (PARTITION BY ...)`) — em muitos casos essas alternativas geram planos mais eficientes para "top N por grupo".

14. Cenário Integrado: Análise Fiscal Completa

O exemplo a seguir combina várias técnicas deste módulo em uma única rotina analítica típica de auditoria fiscal:

```

WITH media_por_regime AS (
    SELECT
        c.regime_tributario,
        AVG(n.valor_total) AS media_regime
    FROM dbo.contribuinte AS c
    JOIN dbo.nfe AS n
        ON n.id_emitente = c.id_contribuinte
    GROUP BY c.regime_tributario
)
SELECT
    c.razao_social,
    c.regime_tributario,
    ultima_nota.chave_acesso      AS ultima_nota_emitida,
    ultima_nota.valor_total      AS valor_ultima_nota,
    mr.media_regime
FROM dbo.contribuinte AS c
-- subconsulta correlacionada via APPLY: última nota de cada contribuinte
OUTER APPLY (
    SELECT TOP (1) n.chave_acesso, n.valor_total
    FROM dbo.nfe AS n
    WHERE n.id_emitente = c.id_contribuinte
    ORDER BY n.data_emissao DESC
) AS ultima_nota
-- CTE reaproveitada para trazer a média do regime do contribuinte
JOIN media_por_regime AS mr
    ON mr.regime_tributario = c.regime_tributario
WHERE EXISTS (
    -- subconsulta correlacionada: só contribuintes com nota acima da média do
    próprio regime
    SELECT 1
    FROM dbo.nfe AS n2
    WHERE n2.id_emitente = c.id_contribuinte
        AND n2.valor_total > mr.media_regime
)
AND c.id_contribuinte NOT IN (
    -- subconsulta independente, com filtro de NULL por segurança
    SELECT id_emitente
    FROM dbo.nfe
    WHERE situacao = 'CANCELADA'
        AND id_emitente IS NOT NULL
)
ORDER BY mr.media_regime DESC;

```

Essa consulta responde: *"quais contribuintes, que nunca tiveram nota cancelada, emitiram ao menos uma nota acima da média do próprio regime tributário — e qual foi a última nota de cada um?"* — combinando CTE, **OUTER APPLY**, subconsulta correlacionada com **EXISTS** e subconsulta independente com **NOT IN** protegida contra **NULL**.

15. Boas Práticas

- Prefira **EXISTS/NOT EXISTS** a **IN/NOT IN** quando a coluna testada puder conter **NULL** — especialmente em **NOT IN** (ver [seção 11](#)).
- Garanta que subconsultas escalares usadas com **=** **sempre** retornem no máximo uma linha (**TOP (1)** com **ORDER BY** determinístico, ou agregação).
- Dê **alias explícito e descritivo** a toda tabela derivada (**FROM (...) AS alias**) — o T-SQL exige o alias, mas um nome significativo também documenta a intenção da subconsulta.
- Evite mais de dois ou três níveis de subconsultas aninhadas: prefira decompor em CTEs nomeadas, que tornam a lógica sequencial e mais fácil de revisar em auditoria de código.
- Ao usar subconsultas correlacionadas em rotinas que rodam sobre tabelas fiscais volumosas, **valide o plano de execução** e garanta índice de apoio na coluna de correlação.
- Prefira **CROSS APPLY/OUTER APPLY** quando a "subconsulta por linha" precisar retornar mais de um valor ou mais de uma linha (ex.: "as 3 notas mais recentes de cada contribuinte").
- Teste sempre o caso de **conjunto vazio** ao usar **ALL** (a condição se torna **TRUE**) e **ANY/SOME** (a condição se torna **FALSE**) — comportamento fácil de esquecer e que já causou falsos positivos em relatórios fiscais.

16. Glossário

Termo	Definição
Subconsulta (subquery)	Instrução SELECT aninhada dentro de outra instrução SQL
Consulta externa / interna	A instrução que contém a subconsulta (externa) e a subconsulta propriamente dita (interna)
Subconsulta escalar	Subconsulta que retorna um único valor (uma linha, uma coluna)
Subconsulta correlacionada	Subconsulta que referencia colunas da consulta externa, sendo logicamente reavaliada a cada linha candidata
Tabela derivada (derived table)	Subconsulta usada na cláusula FROM , tratada como uma tabela temporária com alias obrigatório
APPLY (CROSS/OUTER)	Extensão T-SQL que junta cada linha externa ao resultado de uma subconsulta correlacionada, mesmo que ela retorne múltiplas linhas/colunas
Lógica de três valores	Modelo lógico do SQL em que uma comparação pode resultar em TRUE , FALSE ou UNKNOWN (quando envolve NULL)
Plano de execução	Representação, gerada pelo otimizador, de como o SQL Server efetivamente executará uma consulta (operadores como <i>Seek</i> , <i>Scan</i> , <i>Nested Loops</i> , <i>Hash Match</i>)

17. Exercícios

Parte I — Subconsultas independentes e escalares

1. Liste os contribuintes cujo **id_contribuinte** está entre os que emitiram notas com **valor_total** maior que R\$ 20.000,00, usando **IN**.

2. Escreva uma consulta que traga, para cada nota fiscal, o valor da nota e a média geral de todas as notas em uma coluna calculada (subconsulta escalar no **SELECT**).
3. Encontre o contribuinte com a nota fiscal de maior valor usando uma subconsulta escalar com **TOP (1)** no **WHERE**.

Parte II — **ANY**, **ALL**, **EXISTS**

4. Liste os contribuintes que emitiram alguma nota com valor maior que **qualquer** nota do regime "MEI" (**> ANY**).
5. Liste os contribuintes que emitiram uma nota com valor maior que **todas** as notas do regime "MEI" (**> ALL**).
6. Reescreva o exercício 1 usando **EXISTS** em vez de **IN**, com subconsulta correlacionada.
7. Liste os contribuintes que nunca emitiram nenhuma nota fiscal, usando **NOT EXISTS**.

Parte III — Derived tables, **HAVING** e **APPLY**

8. Construa uma tabela derivada que agregue o total de notas por contribuinte, e filtre no **WHERE** externo apenas os contribuintes com mais de 5 notas.
9. Escreva uma consulta com **HAVING** que traga apenas os regimes tributários cuja soma de **valor_total** supera a soma média entre todos os regimes.
10. Utilize **CROSS APPLY** para trazer, para cada contribuinte, as 2 notas fiscais de maior valor.
11. Utilize **OUTER APPLY** para trazer a última nota emitida por contribuinte, preservando contribuintes sem nenhuma nota.

Parte IV — Armadilhas e comparação

12. Demonstre, com uma massa de dados de teste contendo ao menos um **NULL** em **id_destinatario**, o problema do **NOT IN** descrito na [seção 11](#). Em seguida, corrija a consulta com **NOT EXISTS**.
13. Reescreva a consulta do exercício 7 usando **LEFT JOIN ... WHERE ... IS NULL** e compare o plano de execução com a versão **NOT EXISTS**.

Desafio

Desenvolva uma consulta única (pode usar CTE) que responda: *"Quais contribuintes emitiram, no último trimestre, uma nota com valor acima da média do próprio regime tributário, considerando apenas notas não canceladas, e qual foi a data da nota mais recente de cada um?"*

Requisitos:

- Ao menos uma subconsulta correlacionada.
- Ao menos um uso de **CROSS APPLY** ou **OUTER APPLY**.
- Tratamento seguro de **NULL** (sem uso de **NOT IN** desprotegido).
- Apresente o plano de execução (**Ctrl+M**) e identifique o operador mais custoso.

18. Referências

- **Microsoft Learn — Subqueries (Transact-SQL)**: referência oficial sobre sintaxe e regras de subconsultas no SQL Server (docs.microsoft.com/sql/t-sql/queries).

- **Microsoft Learn — Use Correlated Subqueries:** documentação específica sobre subconsultas correlacionadas.
- **Microsoft Learn — APPLY (Transact-SQL):** referência de **CROSS APPLY/OUTER APPLY**.
- **Microsoft Learn — EXISTS (Transact-SQL) e IN (Transact-SQL):** sintaxe e semântica detalhada de cada operador.
- **Itzik Ben-Gan — "T-SQL Querying"** (Microsoft Press): capítulo dedicado a subconsultas, **APPLY** e lógica de três valores, com discussão aprofundada da armadilha do **NOT IN/NULL**.
- Consulte também [00 Comandos SQL.md](#) para revisão das cláusulas DQL básicas antes deste módulo.